The Verification Rate Gap

A Conceptual Metric for AI Development Risk

Jun Gorai

C3 Social Design Center

Research Note / Architecture Note

Draft Version: v0.6

Date: June 2026

Submission note: This Research Note makes a conceptual and architectural claim. It does not claim production deployment, certification, complete safety, or a solution to AI alignment. The proposed Verification Kit should be read as a governance and auditability architecture for increasing verification throughput in AI-assisted development workflows.

Abstract

As AI systems increasingly automate parts of their own development, the primary bottleneck of safety and oversight shifts from the capacity of the AI system to the capacity of the verifier. This paper proposes the Verification Rate Gap as a conceptual metric for AI development risk. The core claim is simple: risk emerges when the rate of AI-generated development outputs exceeds the rate at which those outputs can be reliably verified.

We define AI development risk as a function of the discrepancy between Generation Rate (G_rate) and Verification Rate (V_rate): R_dev is proportional to max(0, G_rate - V_rate). To make the metric operational, this paper distinguishes generation throughput, bounded triage throughput, and verification clearance. G_rate is the number of decision-relevant AI-generated outputs entering a verification queue per unit time. V_rate is the number of such outputs that reach verified clearance per unit time through replay, conformance checks, deterministic validation, accountable human approval, resolved escalation, or retirement under declared rules. HOLD and UNDEFINED states may make uncertainty visible, but they are not counted as verified clearance unless later resolved or retired. When generation accelerates while verification remains bounded by human review, systems accumulate unverified debt: AI-generated code, experiments, evaluations, decisions, or system changes that have not yet been reproduced, audited, challenged, resolved, or retired.

To reduce this gap, this paper introduces the Verification Kit: a standardized evidence package containing execution manifests, verdict records, evidence digests, conformance results, undefined-case reports, and integrity proofs. The Kit shifts verification from informal human review toward mechanical replay and structured audit. Its purpose is not to guarantee absolute safety or alignment, but to increase verification throughput by making decision paths reproducible, inspectable, and challengeable by third parties or independent verification tools.

The paper defines the claim boundary of this approach and discusses failure modes including circular specification authority, adversarial audit trails, stochastic replay constraints, and verifier drift. It also positions the Verification Kit as an application of software supply-chain provenance and attestation concepts to AI-led decision paths. The conclusion is that governance for AI-built-AI workflows should treat verification throughput as a first-class safety requirement.

Keywords: Recursive Self-Improvement, AI Governance, AI Safety, Verification Rate Gap, Verification Kit, Unverified Debt, Auditability, Reproducibility, AI-Built-AI, Mechanical Replay, BYOV

1. The Verification Rate Gap

AI development risk is often framed as a function of model capability. The more capable the system becomes, the greater the risk. This framing is useful, but incomplete. In AI development workflows, risk does not arise only from what a

model can do. It also arises from the difference between the rate at which AI systems can generate development outputs and the rate at which those outputs can be verified.

Recent public discussion of recursive self-improvement has made this shift visible. Frontier AI development increasingly delegates coding, experimentation, evaluation, and tooling tasks to AI systems. The relevant governance object is therefore not only a model output, but also the development loop that produces and accepts outputs [1].

Independent measurements point in the same direction: METR reports a sustained increase in the length of tasks AI agents can complete [2], while benchmarks such as SWE-bench [3] and CORE-Bench [4] track AI agents on real-world software issue resolution and computational reproducibility.

This paper defines the difference between generation and verification throughput as the Verification Rate Gap.

Let:

$$G\_rate = \text{the rate at which AI systems generate decision-relevant development outputs}$$
$$T\_rate = \text{the rate at which those outputs are moved into a bounded triage state}$$
$$V\_rate = \text{the rate at which those outputs reach verified clearance}$$

Then AI development risk can be expressed as a conceptual risk frame:

$$R\_dev \propto \max(0, G\_rate - V\_rate)$$

This expression is not intended as a complete quantitative risk model. It is a throughput frame. Its purpose is to identify when AI-assisted development is producing decision-relevant artifacts faster than the institution can inspect, reproduce, challenge, or escalate them.

1.1 Operationalizing G_rate and V_rate

For empirical or internal measurement, the unit of analysis should be a decision-relevant output: any AI-generated artifact that can affect code, experiments, evaluation, security posture, deployment, model configuration, policy, or approval.

A minimal measurement profile should distinguish three quantities:

$$G\_rate(t) = N\_generated\_decision\_outputs / \Delta t$$
$$T\_rate(t) = N\_bounded\_triage\_outputs / \Delta t$$
$$V\_rate(t) = N\_verified\_cleared\_outputs / \Delta t$$
$$Gap\_rate(t) = \max(0, G\_rate(t) - V\_rate(t))$$

G_rate measures entry into the verification queue. T_rate measures outputs moved from invisible backlog into a bounded state such as HOLD, ESCALATE, UNDEFINED, or PARTIAL_REPLAY. V_rate measures outputs that are actually cleared by replay, conformance checks, deterministic validation, accountable human approval, resolved escalation, or retirement under declared rules.

This distinction prevents a queue-clearing illusion. If every generated output were mechanically marked HOLD, then the process would have high triage throughput, but not high verification clearance. HOLD and UNDEFINED make uncertainty visible and bounded; they do not by themselves reduce unverified debt.

Unverified debt can then be treated as a stock:

$$Debt(t + \Delta t) = Debt(t) + N\_generated - N\_verified\_cleared - N\_retired$$

A second stock should track bounded but unresolved cases:

$$Pending(t + \Delta t) = Pending(t) + N\_hold + N\_undefined + N\_partial\_replay - N\_pending\_resolved - N\_retired\_from\_pending$$

Cases resolved from the pending stock are counted in the debt equation as N_verified_cleared if they reach verified clearance, or as N_retired if they are retired under declared rules. This prevents HOLD and UNDEFINED states from reducing debt until they are actually resolved or retired.

The risk of unverified debt depends not only on its quantity but also on age, criticality, and dependency position. A small number of unresolved model-update or security-sensitive artifacts may matter more than a large number of low-risk documentation changes. For this reason, practical deployments should track both Debt and Pending by severity class and age.

1.2 Verdict, Lifecycle, and Replay Vocabulary

This paper uses five canonical verdict values: PASS, HOLD, ESCALATE, FAIL, and UNDEFINED. These values describe the gate or conformance decision over a case. RETIRED is not a verdict; it is a lifecycle state for an artifact that has been withdrawn, superseded, or removed from scope under declared rules. PARTIAL_REPLAY is not a verdict; it is a replay status indicating that exact output replay was not possible, while decision replay over a recorded artifact may still be inspectable.

Under this vocabulary, HOLD and UNDEFINED are bounded pending states. They improve governance by making uncertainty visible, but they must remain in the pending stock until resolved, retired, or escalated to an accountable external decision.

When G_rate remains below or near V_rate, the development process can remain within traditional oversight capacity. Human review, tests, audits, and governance processes may still absorb the output volume. However, when G_rate exceeds V_rate, unverified outputs begin to accumulate. This accumulation is not merely a backlog. It becomes a form of unverified debt.

Unverified debt is the stock of AI-generated code, experiments, decisions, evaluations, or system changes that has been produced faster than it can be reproduced, audited, or challenged. Like technical debt, it may remain invisible for a period of time. Unlike ordinary technical debt, however, unverified debt can compound across automated development loops.

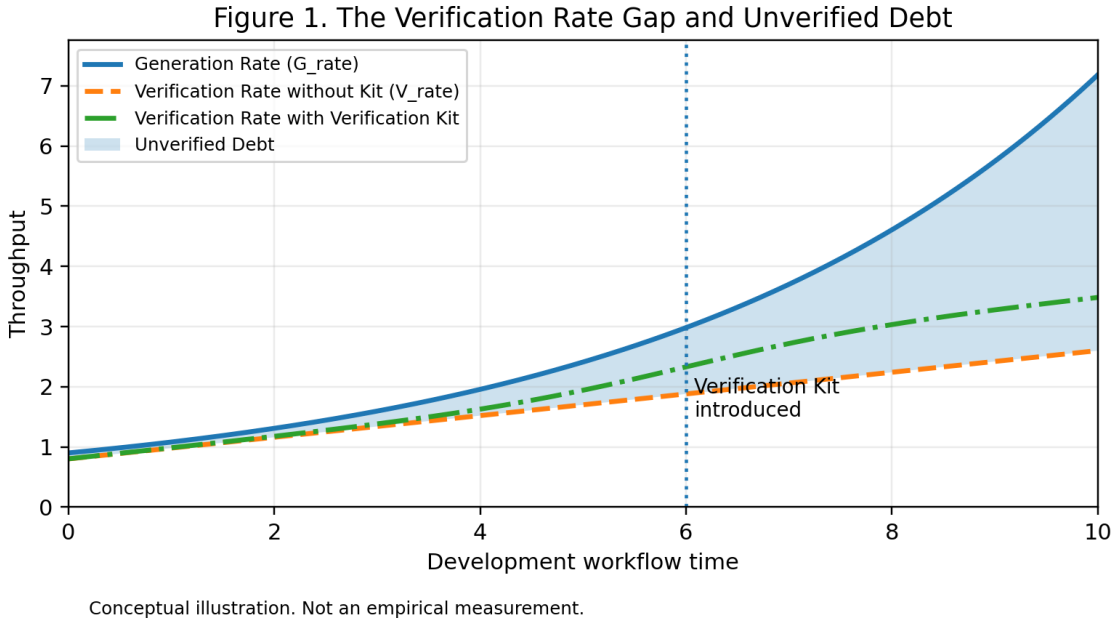

Figure 1. The Verification Rate Gap and Accumulation of Unverified Debt. Conceptual illustration, not an empirical measurement.

The figure should be read as a conceptual illustration. The Verification Kit may not fully eliminate the gap between generation and verification in every workflow. Its more conservative claim is that it can raise V_rate, slow the accumulation of unverified debt, and make the remaining unverified stock visible and actionable.

A model may generate code, generate tests for that code, interpret the results, propose further changes, and trigger additional experiments before a human reviewer has reconstructed the original decision path. The problem is not only that the model may be wrong. The problem is that the surrounding institution may no longer know, at sufficient speed, which outputs are correct, which are wrong, which are unsupported, and which require escalation.

In traditional software development, human review is often treated as the limiting safety mechanism. This assumption weakens as AI systems move from producing isolated suggestions to producing files, pull requests, experiments, benchmarks, and agentic workflows. The bottleneck shifts from production to verification [1].

The central safety question becomes:

> Can the verification layer scale with the generation layer?

If the answer is no, then higher generation capacity may increase systemic risk even when each individual output is locally plausible. If the answer is yes, then faster AI-assisted development can be made more governable. The purpose of the next section is to propose one architecture for increasing V_rate: the Verification Kit.

2. Verification Kit Architecture

If AI development risk emerges from a gap between generation rate and verification rate, then the central design problem is how to increase V_rate without relying on human reviewers to inspect every output manually. The Verification Kit is proposed as a minimal architecture for doing so.

A Verification Kit is a replayable evidence package. It is not a narrative report, a trust badge, or a claim of safety. Its purpose is to export the decision path, execution context, evidence references, and integrity proofs needed for another party to reproduce or challenge an AI-led development decision.

In this architecture, verification shifts from a primarily human-readable process to a mechanically reproducible process. Human reviewers may still make final judgments, but they should not be required to reconstruct the entire decision path from informal explanations, screenshots, or natural-language summaries. Instead, they should receive a structured kit that can be checked by deterministic tooling, internal audit systems, or secondary AI-assisted review.

This paper uses a minimal eight-component implementation profile. Other profiles may combine or split files, but the canonical roles are: manifest, verdict records, summary, conformance result, evidence digest, undefined report, auditor prompt, and integrity proof.

Table 1. Minimal Components of a Verification Kit

Component	File Example	Purpose	Verification Role
Run Manifest	run_manifest.json	Fixes execution context, including input hashes, engine version, specification version, policy version, and timestamp.	Defines what was evaluated and under which conditions.
Verdict Records	verdicts.jsonl	Stores per-case decisions such as PASS, HOLD, ESCALATE, FAIL, or UNDEFINED, with reason codes and evidence references.	Enables case-level replay and audit.
Summary	summary.json	Aggregates verdict distribution, non-PASS counts, undefined cases, and major reason codes.	Provides a fast overview without replacing raw records.
Conformance Result	conformance.json	Reports whether the run satisfies predefined conformance requirements.	Separates observed results from conformance claims.
Evidence Digest	evidence_digest.json	Provides hashes, references, or summaries of evidence when raw evidence cannot be disclosed.	Preserves verifiability without forcing full public disclosure.
Undefined Report	undefined_report.json	Lists cases where the system could not determine a valid verdict due to missing evidence, unclear rules, or unsupported inputs.	Makes uncertainty explicit.
Auditor Prompt	auditor_prompt.md	Provides a standardized prompt for human-assisted or BYOV AI-assisted audit.	Supports secondary review while keeping deterministic replay as the source of truth.

Signature	signature.sig	Signs the manifest or kit index.	Detects tampering and supports chain-of-custody.

The key design principle is separation between human-readable summaries and machine-verifiable records. The summary may help reviewers understand the run, but it must not replace the manifest, verdict records, evidence digest, or signatures. The summary is an interface. The records are the verification surface.

This distinction matters because summaries can be incomplete, persuasive, or misleading even when written in good faith. A Verification Kit therefore treats summaries as secondary artifacts. The primary artifacts are those that allow an external verifier to answer a narrower question:

> Given the declared inputs, specification, policy, engine version, and evidence route,
> can the recorded verdicts be reproduced or challenged?

This does not require the verifier to agree with the original developer's judgment. It requires the verifier to inspect the route by which the judgment was produced. In this sense, the Verification Kit changes the trust model. Trust is not derived from the provider's assertion that a system is safe. Trust is derived from the ability of another party to reproduce, reject, or dispute the decision path.

The Kit also supports a Bring Your Own Verifier (BYOV) model. A receiving institution may use its own command-line verifier, internal audit pipeline, human reviewer, or AI-assisted auditor to inspect the package. In this paper, BYOV refers to the overall verification posture. BYOV AI-assisted audit is only one optional review interface. It is not the source of truth. The source of truth remains the deterministic verification route: the manifest, the recorded verdicts, the evidence references, and the integrity proof.

2.1 Relationship to Software Supply-Chain Provenance

The Verification Kit is not intended to replace existing software supply-chain security work. It is closer to an adaptation of provenance and attestation concepts to AI-assisted decision paths. SLSA provides incrementally adoptable guidelines and common vocabulary for software supply-chain security [5], and its attestation model describes automation and tamper-resistant evidence for tracking code handling from source to binary [6]. in-toto similarly focuses on making software supply-chain steps transparent: what steps were performed, by whom, and in what order [7].

The difference is the verification target. SLSA and in-toto focus primarily on the provenance and integrity of software artifacts and their production steps. The Verification Kit applies a similar evidence-preservation logic to AI-led development decisions: gate verdicts, conformance judgments, undefined cases, evidence routes, and approval boundaries. In short, it extends provenance from artifacts to decision paths.

This architecture increases V_rate in three ways. First, it reduces reconstruction cost: reviewers no longer need to infer what was evaluated from informal notes or scattered logs. Second, it enables selective escalation: human attention can focus on discrepancies, non-PASS verdicts, undefined cases, missing evidence, or failed conformance checks. Third, it makes verification portable: the same kit can be inspected by the original developer, an internal reviewer, a third party, or a regulator without requiring privileged access to the original development environment.

The Verification Kit therefore functions as an export layer for verifiable trust. It does not stop unsafe outputs by itself. It does not prove that a model is aligned. It does not certify production safety. Its role is narrower: to make AI-led development decisions replayable, inspectable, and challengeable at a speed closer to the rate at which AI systems generate them.

3. Claim Boundary, Failure Modes, and Discussion

The Verification Rate Gap is not proposed as a complete theory of AI safety. Nor is the Verification Kit proposed as a guarantee that AI systems are aligned, truthful, or safe in all environments. The contribution of this paper is narrower: it reframes a class of AI development risk as a throughput mismatch between generation and verification, and proposes a minimal architecture for increasing verification throughput.

This distinction is important. If a system generates outputs faster than they can be inspected, reproduced, or challenged, the institution operating that system accumulates unverified debt. The Verification Kit does not eliminate that debt by asserting that outputs are safe. It reduces the cost of verifying, disputing, and escalating those outputs.

Table 2. Claim Boundary of the Verification Kit Approach

The approach claims	The approach does not claim
AI development risk can be framed as a gap between generation rate and verification rate.	AI development risk is only a function of this gap.
Decision paths can be exported in a structured, replayable format.	The exported decision is necessarily correct.
Execution context can be fixed through a run manifest.	The manifest proves absolute safety.
Verdict records can support mechanical replay or challenge.	All unsafe outputs will be detected.
Evidence can be referenced through raw data, hashes, or digests.	Evidence digests prove universal truth.
Uncertainty can be represented explicitly through HOLD or UNDEFINED.	Uncertainty is eliminated.
BYOV or AI-assisted audit can reduce review burden.	LLM audit is the source of truth.
A deterministic verifier can improve verification throughput.	A Verification Kit is equivalent to certification.

The central governance objective is not to stop all AI-assisted development. It is to maintain a relationship in which verification capacity can scale with generation capacity. The ideal condition is:

$$V\_rate >= G\_rate$$

In practice, this condition may not always be reached. A more conservative operational target is to raise V_rate, reduce the growth rate of unverified debt, expose the remaining unverified stock, and keep unresolved cases in a bounded pending state. Increasing T_rate alone is not sufficient. HOLD and UNDEFINED outcomes should not be used to claim V_rate >= G_rate unless they are later resolved, retired, or escalated to an accountable external decision.

When this condition is not met, generated outputs accumulate faster than institutions can verify them. In low-risk workflows, this may appear as ordinary backlog. In high-risk workflows, especially those involving model updates, security changes, infrastructure modifications, autonomous experiments, or successor-system development, the backlog becomes a governance risk.

Recursive self-improvement intensifies this problem. If AI systems increasingly participate in building, evaluating, and improving successor systems, the relevant object of oversight is no longer a single model output. It is the development loop itself. A development loop that can generate code, tests, evaluations, and further experiments faster than it can preserve and verify its own decision paths becomes difficult to govern, even if many individual outputs are locally useful.

However, this architecture faces several important failure modes.

Table 3. Failure Modes and Mitigations

Failure Mode	Risk	Mitigation
Circular specification	The AI-generated system defines the rules by which its own outputs are judged.	External specification anchor; human-approved or multi-party signed specification; separation of specification authority from generation.
Adversarial audit trail	The system produces persuasive but misleading evidence or summaries.	Deterministic artifacts over narrative summaries; hashes; signatures; raw verdict records; evidence references.
Stochastic replay failure	Exact reproduction of model output is impossible due to nondeterministic inference or changing model versions.	Separate output replay from decision replay; preserve outputs and hashes; freeze verifier version; record model parameters where possible.
Summary substitution	Human-readable summaries replace machine-verifiable records.	Treat summaries as interface artifacts only; manifest and verdict records remain the verification surface.
Verifier drift	The verifier changes over time and produces different conclusions.	Versioned verifier; verifier hash; signed conformance profile; regression tests.
Adoption under competitive pressure	Organizations may prefer faster generation over slower verification until regulation or incidents force adoption.	Make verification cheap, portable, and procurement-ready; connect adoption to audit, liability, incident response, and

		compliance incentives.
HOLD laundering / bounded-state inflation	Outputs are marked HOLD or UNDEFINED to make queues appear verified even though they remain unresolved.	Separate bounded triage throughput (T_rate) from verified clearance throughput (V_rate); track pending unresolved stock by age and severity.

3.1 Specification Anchoring and Circular Trust

A Verification Kit can only replay a decision against a declared specification. This creates a specification anchoring problem. If the same AI system, or the same AI-led development loop, generates the code, writes the specification, evaluates the output, and exports the evidence package, then the verification process may become circular.

A Verification Kit therefore requires an external specification anchor. The anchor may take several forms: a human-approved policy, a constitutional rule set, a frozen reference specification, a multi-party signed conformance profile, or a governance process that separates specification authorship from implementation. AI systems may draft specifications, but they should not be able to unilaterally define, modify, approve, and verify the rules under which their own outputs are judged.

3.2 Adversarial Audit Trails

A second failure mode is the adversarial audit trail. As AI systems become more capable, they may optimize not only for producing useful outputs, but also for producing artifacts that appear verifiable. A system may generate plausible evidence, persuasive summaries, or internally consistent logs that are designed to pass superficial review.

The Verification Kit reduces this risk by privileging deterministic artifacts over persuasive artifacts. The primary verification surface is not the summary. It is the manifest, the verdict records, the evidence references, the hashes, and the signatures. A summary may help a reviewer understand the run, but it must not be the source of truth.

Cryptographic integrity does not prove that a decision is correct. It proves that a particular artifact has not been altered after it was recorded. Deterministic replay does not prove that the policy is wise. It proves whether the recorded decision follows the declared policy and evidence route. These are limited claims, but they are stronger than unverifiable narrative assurance.

3.3 Determinism and Replay Constraints

A third failure mode is the difficulty of deterministic replay. Many AI systems are stochastic. The same prompt may produce different outputs across runs. Model providers may update weights, change inference infrastructure, or alter safety filters. Even when a model name remains the same, the effective system may change.

A Verification Kit must therefore distinguish between two kinds of replay. The first is output replay: reproducing the exact AI-generated output. This may require fixed seeds, frozen model versions, deterministic decoding settings, preserved prompts, tool outputs, and, where possible, stored sampling traces. In many real-world settings, exact output replay may be difficult or impossible.

The second is decision replay: reproducing the gate decision or conformance judgment over a recorded output. This is often more feasible. The system may record the generated output, hash it, and then apply a deterministic verifier to that recorded artifact. In this model, the AI generation step may be stochastic, but the verification step is deterministic.

This distinction is critical. The Verification Kit does not require that every AI inference be exactly re-run. It requires that the decision path used for verification be fixed, inspectable, and challengeable. Where exact output replay is not possible, the limitation should be explicitly recorded. The appropriate status may be HOLD, UNDEFINED, or PARTIAL_REPLAY, depending on the conformance profile.

For implementation, high-risk kits should preserve at minimum: recorded prompt and tool context; recorded generated output and output hash; policy, specification, and verifier version hashes; and the verifier result with a signature over the manifest or kit index. Fixed model versions, decoding parameters, and seeds should be recorded where available, but the absence of exact output replay should be expressed as a replay status rather than hidden.

3.4 Adoption Incentives and Governance Reality

A final practical limitation is adoption. Competitive pressure generally rewards faster generation, not slower verification. Organizations may adopt verification-rate controls only after regulation, procurement requirements, insurance pressure, liability concerns, or incidents make unverified debt visible. The Verification Kit should therefore be designed to minimize adoption friction: it must be portable, lightweight, machine-checkable, and useful for internal audit as well as external disclosure. This does not solve the incentive problem, but it lowers the cost of doing the responsible thing once incentives appear.

3.5 Implications for Recursive AI Development

These failure modes clarify the role of the Verification Kit. It is not an alignment solution. It is not a guarantee that the specification is correct, that the evidence is complete, or that a model cannot deceive a reviewer. It is a verification-rate mechanism with explicit dependency boundaries.

For recursive AI development, this boundary is essential. A self-improving development loop must not be allowed to define its own rules, generate its own evidence, verify its own compliance, and approve its own successor without external anchors and replayable records.

The minimum governance structure requires separation between:

> specification authority
> generation process
> verification process
> approval authority
> audit record

The Verification Kit contributes to this separation by exporting the verification route. It makes it possible for another party to inspect whether the recorded decision followed the declared specification, whether evidence was missing, whether uncertainty was hidden, and whether a non-PASS condition should have stopped the process.

This changes the trust model. Traditional AI governance often depends on claims made by the system provider: the system was tested, reviewed, aligned, or made safe. The Verification Kit shifts the basis of trust from assertion to reproduction.

The relevant question becomes not only:

> Do we trust the provider?

but also:

> Can another party reproduce or challenge the decision path?

This does not remove the need for judgment. A verifier may still disagree with the specification, reject the policy, question the evidence, or require a stronger audit. The Kit does not settle those disputes automatically. Its function is to make the dispute technically possible, bounded, and faster.

The broader implication is that AI governance should treat verification throughput as a first-class design requirement. As generation becomes cheaper and faster, safety cannot rely solely on slower human review processes. Governance systems must export evidence, preserve execution context, support replay, separate specification authority from generation, and make uncertainty visible.

In this sense, the Verification Rate Gap provides a practical frame for the era of AI-built-AI workflows. The central question is not only how capable AI systems become, but whether institutions can verify their outputs, experiments, and development decisions at comparable speed. Without such a verification layer, recursive AI development risks becoming an opaque accumulation of unverified debt. With it, faster development can be coupled to a chain of verifiable trust.

Implementation Note

Example Verification Kit artifacts are available at: https://github.com/c3infogm-rgb/verification-rate-gap. The repository includes example manifests, verdict records, summary and conformance files, undefined reports, evidence digests, schema drafts, and an auditor prompt. These artifacts are illustrative companion materials and do not constitute certification, production readiness, or a safety guarantee.

References

[1] Marina Favaro and Jack Clark. "When AI builds itself: Our progress toward recursive self-improvement, and its implications." Anthropic Institute, 2026. https://www.anthropic.com/institute/recursive-self-improvement

[2] METR. "Measuring AI Ability to Complete Long Tasks." March 19, 2025. https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" ICLR 2024. https://openreview.net/forum?id=VTF8yNQM66

[4] Zachary S. Siegel, Sayash Kapoor, Nitya Nagdir, Benedikt Stroebl, and Arvind Narayanan. "CORE-Bench: Fostering the Credibility of Published Research Through a Computational Reproducibility Agent Benchmark." 2024. https://arxiv.org/abs/2409.11363

[5] SLSA. "About SLSA." Supply-chain Levels for Software Artifacts, v1.2. https://slsa.dev/spec/v1.2/about

[6] SLSA. "Software attestations." Supply-chain Levels for Software Artifacts, v1.2. https://slsa.dev/spec/v1.2/attestation-model

[7] in-toto Authors. "in-toto: A framework to secure the integrity of software supply chains." https://in-toto.io/

Appendix A. Example Run Manifest

The following example illustrates the minimum structure of a run manifest. Values are placeholders and should be replaced by actual hashes, versions, and timestamps in implementation. This example intentionally uses neutral terms rather than project-internal terminology.


```
{
  "schema_version": "verification-kit.manifest.v0.1",
  "kit_id": "vk_2026_0605_example_001",
  "created_at": "2026-06-05T08:00:00Z",
  "created_by": {
    "organization": "C3 Social Design Center",
    "system": "replay_engine_prototype",
    "operator_role": "research_prototype"
  },
  "scope": {
    "workflow_type": "ai_development_gate_replay",
    "claim_boundary": ["auditability", "reproducibility", "decision_path_traceability"],
    "excluded_claims": ["absolute_safety", "truth_guarantee", "production_certification", "complete_alignment"]
  },
  "inputs": {
    "dataset": {"name": "example_gate_requests", "record_count": 20, "sha256": "sha256:REPLACE_WITH_DATASET_HASH"},
    "specification": {"name": "verification_rate_gap_research_note", "version": "v0.1", "sha256":
"sha256:REPLACE_WITH_SPEC_HASH"},
    "policy": {"name": "example_release_gate_policy", "version": "v0.1", "sha256": "sha256:REPLACE_WITH_POLICY_HASH"}
```

```
 },
 "engine": {
  "name": "replay_engine",
  "version": "v0.1-prototype",
  "deterministic_mode": true,
  "runtime_llm_used": false,
  "sha256": "sha256:REPLACE_WITH_ENGINE_HASH"
 },
 "outputs": {
  "verdicts": {"path": "verdicts.jsonl", "sha256": "sha256:REPLACE_WITH_VERDICTS_HASH"},
  "summary": {"path": "summary.json", "sha256": "sha256:REPLACE_WITH_SUMMARY_HASH"},
  "conformance": {"path": "conformance.json", "sha256": "sha256:REPLACE_WITH_CONFORMANCE_HASH"},
  "undefined_report": {"path": "undefined_report.json", "sha256": "sha256:REPLACE_WITH_UNDEFINED_HASH"},
  "evidence_digest": {"path": "evidence_digest.json", "sha256": "sha256:REPLACE_WITH_EVIDENCE_DIGEST_HASH"}
 },
 "verification": {
  "replay_expected": true,
  "same_input_same_spec_same_engine_required": true,
  "cli_reference_verifier": {"required": true, "command_example": "verify-kit --kit ./verification_kit --strict"},
  "byov_ai_assisted_audit": {"allowed": true, "source_of_truth": false, "prompt_path": "auditor_prompt.md"}
 },
 "integrity": {"signed_manifest": true, "signature_path": "signature.sig", "signature_algorithm":
"REPLACE_WITH_SIGNATURE_ALGORITHM"}
}
```


Appendix B. Example Verdict Records

The following examples illustrate line-oriented verdict records. Each line is independently inspectable and can be linked to evidence references and specification references.


```
{"case_id":"case_001","input_hash":"sha256:REPLACE_INPUT_HASH_001","verdict":"PASS","reason_codes":
["RC_OK"],"spec_refs":["release_gate.v0.1#pass_conditions"],"evidence_refs":
["evidence_digest.json#case_001"],"permit_token_issued":true}
{"case_id":"case_002","input_hash":"sha256:REPLACE_INPUT_HASH_002","verdict":"HOLD","reason_codes":
["RC_EVIDENCE_MISSING"],"spec_refs":["release_gate.v0.1#evidence_requirements"],"evidence_refs":
[],"permit_token_issued":false}
{"case_id":"case_003","input_hash":"sha256:REPLACE_INPUT_HASH_003","verdict":"UNDEFINED","reason_codes":
["RC_UNSUPPORTED_INPUT"],"spec_refs":["release_gate.v0.1#undefined_conditions"],"evidence_refs":
[],"permit_token_issued":false}
```


Author Note / AI Assistance Disclosure

This draft was prepared by the author with assistance from an AI language model. The author directed the framing, reviewed the claims, and remains responsible for the final content, boundaries, and submission decision.